uses and add middleware that transforms requests or responses. It may not be needed in every app, but it's nice to have full control when you need it.

Next, let's connect this client module to our app.

Use the API Client in Your App

As usual, we're going to expose our functions inside of a context module. Create lib/phone_app/twilio/twilio.ex and add the following delegates:

phone_app/lib/phone_app/twilio/twilio.ex

```
defmodule PhoneApp.Twilio do
  defdelegate send_sms_message!(msg), to: PhoneApp.Twilio.Api
  defdelegate get_sms_message!(msg), to: PhoneApp.Twilio.Api
end
```

Now, we can replace the PhoneApp.Conversations.send_sms_message/1 to deliver a message using our SMS API. Replace the existing function in lib/phone_app/conversations/conversations.ex with the following code:

phone_app/lib/phone_app/conversations/conversations.ex

```
Line 1  def send_sms_message(params = %Schema.NewMessage{}) do
        msg = %{
          from: your_number(),
          to: params.to,
          body: params.body
        }

        case PhoneApp.Twilio.send_sms_message!(msg) do
          %{body: resp = %{}} ->
            params = %{
              message_sid: resp["sid"],
              account_sid: resp["account_sid"],
              body: resp["body"],
              from: resp["from"],
              to: resp["to"],
              status: resp["status"],
              direction: :outgoing
            }

            create_sms_message(params)

          %{body: %{"code" => _, "message" => err}} ->
            {:error, err}

          _err ->
            {:error, "Failed to send message"}
        end
      end
```